COMP-3670 – Fall 2023

Assignment 1: Wireshark Getting Started & Ethernet

Include screenshots of each step (mentioned in each question) with your answers.

Objective

"Wireshark Getting Started" and explore the details of Ethernet frames. Ethernet is a popular link layer protocol. Modern computers connect to Ethernet switches rather than use classic Ethernet. Review related topics before doing this lab.

One's understanding of network protocols can often be greatly deepened by "seeing protocols in action" and by "playing around with protocols" – observing the sequence of messages exchanged between two protocol entities, delving down into the details of protocol operation, and causing protocols to perform certain actions and then observing these actions and their consequences. This can be done in simulated scenarios or in a "real" network environment such as the Internet. In the assignments you'll be doing in this course, you'll observe the network protocols in your computer "in action," interacting and exchanging messages with protocol entities executing elsewhere on the Internet. You'll observe, and you'll learn, by doing.

In the first section, you'll get acquainted with Wireshark, and make some simple packet captures and observations. The second section of this assignment will start to explore protocols more in-depth.

Requirements

The basic tool for observing the messages exchanged between executing protocol entities is called a **packet sniffer**. As the name suggests, a packet sniffer captures ("sniffs") messages being sent/received from/by your computer; it will also typically store and/or display the contents of the various protocol fields in these captured messages. A packet sniffer itself is passive. It observes messages being sent and received by applications and protocols running on your computer, but never sends packets itself. Similarly, received packets are never explicitly addressed to the packet sniffer. Instead, a packet sniffer receives a *copy* of packets that are sent/received from/by application and protocols executing on your machine.

Figure 1 shows the structure of a packet sniffer. At the right of Figure 1 are the protocols (in this case, Internet protocols) and applications (such as a web browser or email client) that normally run on your computer. The packet sniffer, shown within the dashed rectangle in Figure 1 is an addition to the usual software in your computer, and consists of two parts. The **packet capture library** receives a copy of every link-layer frame that is sent from or received by your computer over a given interface (link layer, such as Ethernet or WiFi). Messages exchanged by higher layer protocols such as HTTP, FTP, TCP, UDP, DNS, or IP all are eventually encapsulated in link-layer frames that are transmitted over physical media such as an Ethernet cable or an 802.11 WiFi radio. Capturing all link-layer frames thus gives you all messages sent/received across the monitored link from/by all protocols and applications executing in your computer.

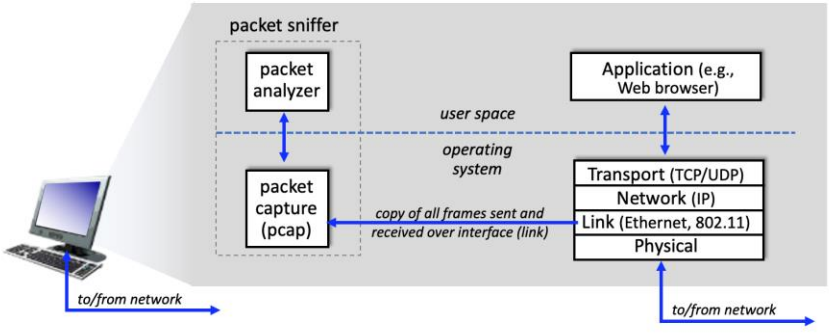


Figure 1: packet sniffer structure

The second component of a packet sniffer is the **packet analyzer**, which displays the contents of all fields within a protocol message. In order to do so, the packet analyzer must "understand" the structure of all messages exchanged by protocols. For example, suppose we are interested in displaying the various fields in messages exchanged by the HTTP protocol in Figure 1. The packet analyzer understands the format of Ethernet frames, and so can identify the IP datagram within an Ethernet frame. It also understands the IP datagram format, so that it can extract the TCP segment within the IP datagram. Finally, it understands the TCP segment structure, so it can extract the HTTP message contained in the TCP segment. Finally, it understands the HTTP protocol and so, for example, knows that the first bytes of an HTTP message will contain the string "GET," "POST," or "HEAD".

1- **Wireshark**: We will be using the Wireshark packet sniffer [http://www.wireshark.org/] for these assignments, allowing us to display the contents of messages being sent/received from/by protocols at different levels of the protocol stack. (Technically speaking, Wireshark is a packet analyzer that uses a packet capture library in your computer. Also, technically speaking, Wireshark captures link-layer frames as shown in Figure 1 but uses the generic term "packet" to refer to link-layer frames, network-layer datagrams, transport-layer segments, and application-layer messages, so we'll use the less-precise "packet" term here to go along with Wireshark convention). Wireshark is a free network protocol analyzer that runs on Windows, Mac, and Linux/Unix computers. It provides a graphical UI that shows the sequence of packets and the meaning of the bits when interpreted as protocol headers and data. It colour-codes packets by their type and has various ways to filter and analyze packets to let you investigate the behavior of network protocols. It's an ideal packet analyzer for our assignments – it is stable, has a large user base and well-documented support that includes a user-guide (http://www.wireshark.org/docs/wsug_html_chunked/), man pages (http://www.wireshark.org/docs/man-pages/), and a detailed FAQ (http://www.wireshark.org/faq.html), rich functionality that includes the capability to analyze hundreds of protocols, and a well-designed user interface. It operates in computers using Ethernet, serial (PPP), 802.11 (WiFi) wireless LANs, and many other link-layer technologies.

We recommend that you watch the videos of "Introduction to Wireshark" too, that is on https://www.youtube.com/playlist?list=PLAqTI-VaLtp0wnX3PgcpgFqLno6_i57kJ

Getting Wireshark

In order to run Wireshark, you'll need to have access to a computer that supports both Wireshark and the *libpcap* or *WinPCap* packet capture library. The *libpcap* software will be installed for you, if it is not installed within your operating system when you install Wireshark. See

http://www.wireshark.org/download.html for a list of supported operating systems and download sites.

Download and install the Wireshark software:

- Go to http://www.wireshark.org/download.html and download and install the Wireshark binary for your computer.

The Wireshark FAQ has a number of helpful hints and interesting tidbits of information, particularly if you have trouble installing or running Wireshark.

Running Wireshark

When you run the Wireshark program, you'll get a startup screen that looks something like the screen below. Different versions of Wireshark will have different startup screens – so don't panic if yours doesn't look exactly like the screen below! The Wireshark documentation states "As Wireshark runs on many different platforms with many different window managers, different styles applied and there are different versions of the underlying GUI toolkit used, your screen might look different from the provided screenshots. But as there are no real differences in functionality these screenshots should still be well understandable." Well said.

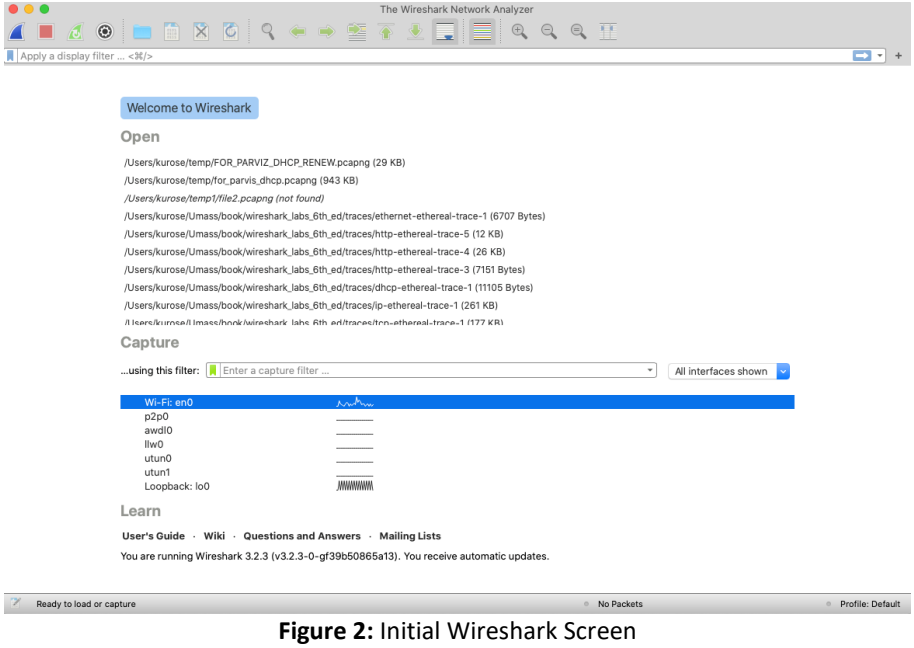


Figure 2: Initial Wireshark Screen

There's not much that's very interesting on this screen. But note that under the Capture section, there is a list of so-called interfaces. The Mac computer we're taking these screenshots from has just one interface – "Wi-Fi en0," (shaded in blue in Figure 2) which is the interface for Wi-Fi access. All packets to/from this computer will pass through the Wi-Fi interface, so it's here where we'll want to capture packets. On a Mac, double click on this interface (or on another computer locate the interface on startup page through which you are getting Internet connectivity, e.g., most likely a WiFi or Ethernet interface, and select that interface).

Let's take Wireshark out for a spin! If you click on one of these interfaces to start packet capture (i.e., for Wireshark to begin capturing all packets being sent to/from that interface), a screen like the one below will be displayed, showing information about the packets being captured. Once you start packet

capture, you can stop it by using the Capture pull down menu and selecting Stop (or by clicking on the red square button next to the Wireshark fin in Figure 2). [1]

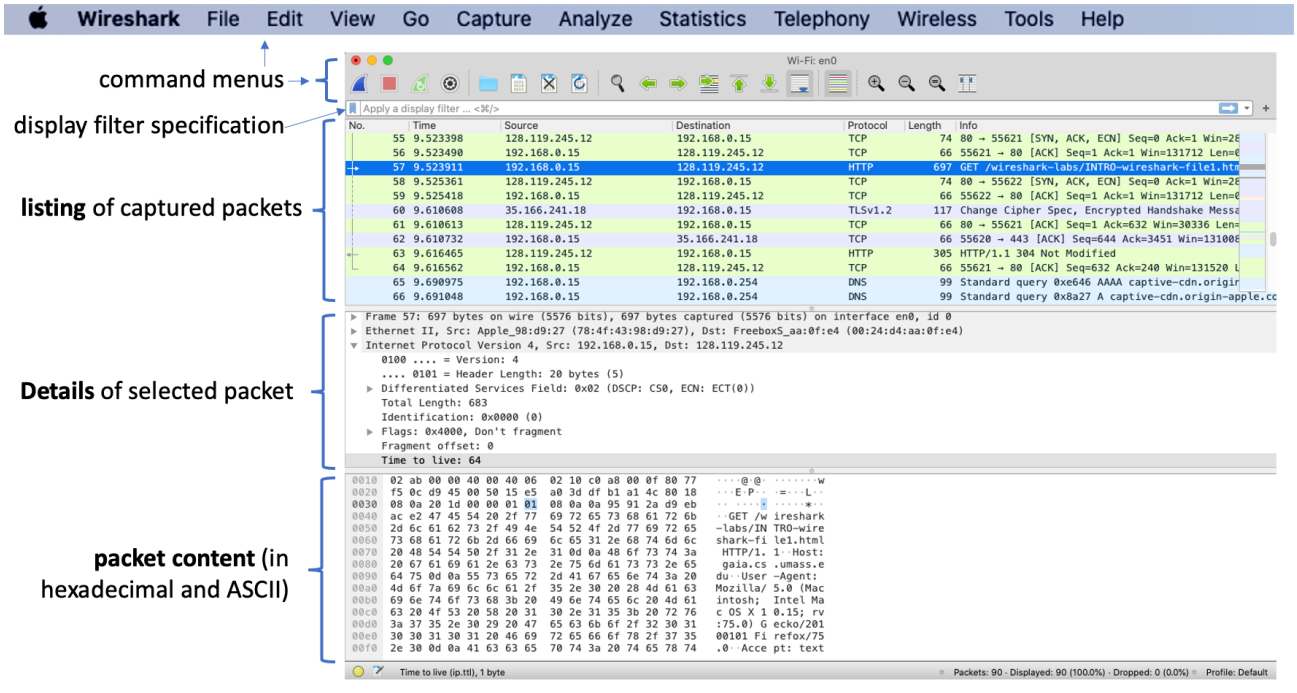


Figure 3: Wireshark window, during and after capture

This looks more interesting! The Wireshark interface has five major components:

- The **command menus** are standard pulldown menus located at the top of the Wireshark window (and on a Mac at the top of the screen as well; the screenshot in Figure 3 is from a Mac). Of interest to us now are the File and Capture menus. The File menu allows you to save captured packet data or open a file containing previously captured packet data and exit the Wireshark application. The Capture menu allows you to begin packet capture.
- The **packet-listing window** displays a one-line summary for each packet captured, including the packet number (assigned by Wireshark; note that this is *not* a packet number contained in any protocol's header), the time at which the packet was captured, the packet's source and destination addresses, the protocol type, and protocol-specific information contained in the packet. The packet listing can be sorted according to any of these categories by clicking on a column name. The protocol type field lists the highest-level protocol that sent or received this packet, i.e., the protocol that is the source or ultimate sink for this packet.

---

[1] If you are unable to run Wireshark, you can still look at prepared packet traces. This lab uses the packet trace captured by Wireshark. A packet trace is a record of traffic at a location on the network as if a snapshot was taken of all the bits that passed across a particular wire. The packet trace records a timestamp for each packet, along with the bits that make up the packet, from the lower-layer headers to the higher-layer contents. Download the zip file http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces-8E.zip and extract the file *wireshark-intro-trace*. The traces in this zip file were collected by Wireshark while performing the steps indicated above. Once you have downloaded the trace, you can load it into Wireshark and view the trace using the *File* pull down menu, choosing *Open*, and then selecting the *wireshark-intro-trace* trace file. The resulting display should look similar to Figures 3. (The Wireshark user interface displays just a bit differently on different operating systems, and in different versions of Wireshark).

- The **packet-header details window** provides details about the packet selected (highlighted) in the packet-listing window. (To select a packet in the packet-listing window, place the cursor over the packet's one-line summary in the packet-listing window and click with the left mouse button.). These details include information about the Ethernet frame (assuming the packet was sent/received over an Ethernet interface) and IP datagram that contains this packet. The amount of Ethernet and IP-layer detail displayed can be expanded or minimized by clicking on the plus/minus boxes or right/downward-pointing triangles to the left of the Ethernet frame or IP datagram line in the packet details window. If the packet has been carried over TCP or UDP, TCP or UDP details will also be displayed, which can similarly be expanded or minimized. Finally, details about the highest-level protocol that sent or received this packet are also provided.
- The **packet-contents window** displays the entire contents of the captured frame, in both ASCII and hexadecimal format.

Towards the top of the Wireshark graphical user interface, is the **packet display filter field,** into which a protocol name or other information can be entered in order to filter the information displayed in the packet-listing window (and hence the packet-header and packet-contents windows).

Question 1 (2 marks): List 4 different protocols that appear in the protocol column in the unfiltered packet-listing window in the step above (figure 3).

2- **ping**: This lab uses "ping" to send and receive messages. ping is a standard command-line utility for checking that another computer is responsive. It is widely used for network troubleshooting and comes pre-installed on Window, Linux, and Mac. While ping has various options, simply issuing the command "ping www.bing.com" will cause your computer to send a small number of ICMP (Internet Control Message Protocol) ping requests to the remote computer (here www.bing.com), each of which should elicit an ICMP ping response.

Taking Wireshark for a Test Run

Step 1: Capture a Trace

Proceed as follows to capture a trace of ping packets; alternatively, you may use a supplied trace. We will use ping simply as an easy way to collect a small trace. Perhaps surprisingly, you can capture a trace for this lab from a computer connected to the Internet using either wired Ethernet or wireless 802.11.

1. *Pick a remote web server or other publicly reachable Internet host and use* ping *to send some ping messages and check that it sends replies*. For example, "ping www.bing.com". You should see several replies indicating that the pings reached the remote host and were returned. Figure 4 shows a successful example. Note that some versions of ping will continue to bounce messages off a remote server until you tell the program to stop by signaling it with ^C. If your ping test does not succeed, then try another server.

Question 2 (1 Mark): Submit a screenshot of your ping. It should look similar to Figure 4.

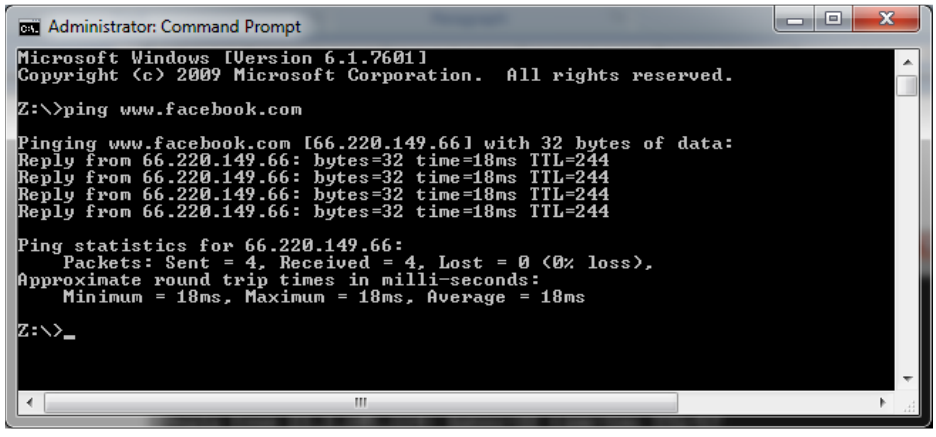
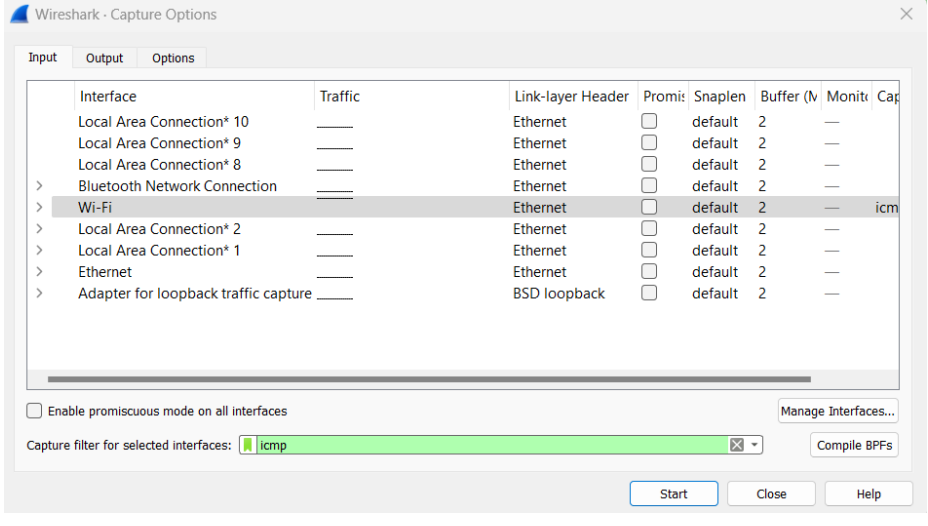
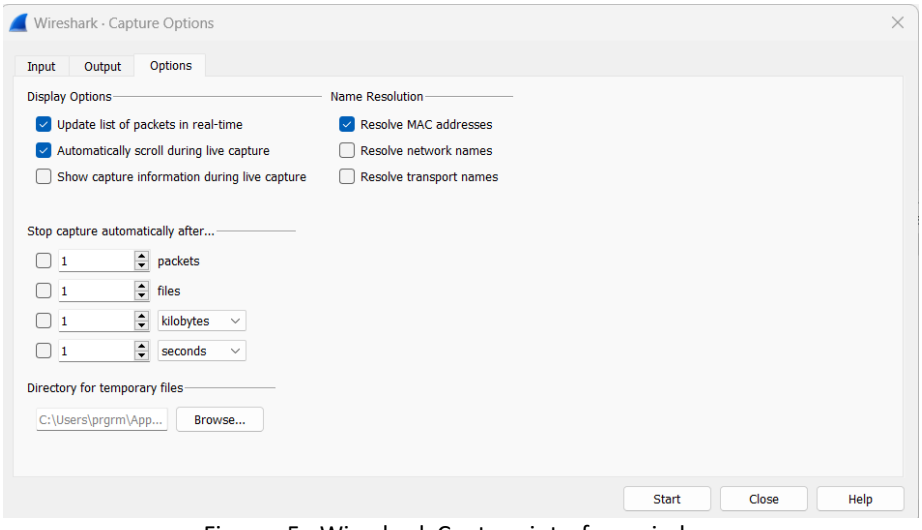


Figure 4: Using ping to bounce messages off a remote host

2. *Launch Wireshark.* To begin packet capture, select *the Capture pull down menu and select Options. This will cause the "Wireshark: Capture Options" window to be displayed (on a PC) or you can choose Options on a Mac. You should see a list of interfaces, as shown in Figure 5. Start a capture of Ethernet frames with a filter of "icmp", making sure that "enable MAC name resolution" is checked in the Option tab. The latter will translate Ethernet (MAC) addresses to provide vendor information. Also check that the Link-layer header type should says "Ethernet". Select the interface from which to capture as the main wired or wireless interface used by your computer to connect to the Internet. If unsure, guess and revisit this step later if your capture is not successful. Uncheck "capture packets in promiscuous mode". This mode is useful to overhear packets sent to/from other computers on broadcast networks. We only want to record packets sent to/from your computer. Leave other options at their default values. The capture filter, if present, is used to prevent the capture of other traffic your computer may send or receive.*





Figures 5: Wireshark Capture interface window

3. *When the capture is started, repeat the* ping *command above.* This time, the packets will also be recorded by Wireshark.
4. *After the* ping *command is complete, return to Wireshark and use the menus or buttons to stop the trace.* You should now have a short trace similar to that shown in the figure below. If you do not succeed in capturing a trace then use the supplied one. Note that the trace we supply begins with ping messages, and then has other kinds of Ethernet frames.

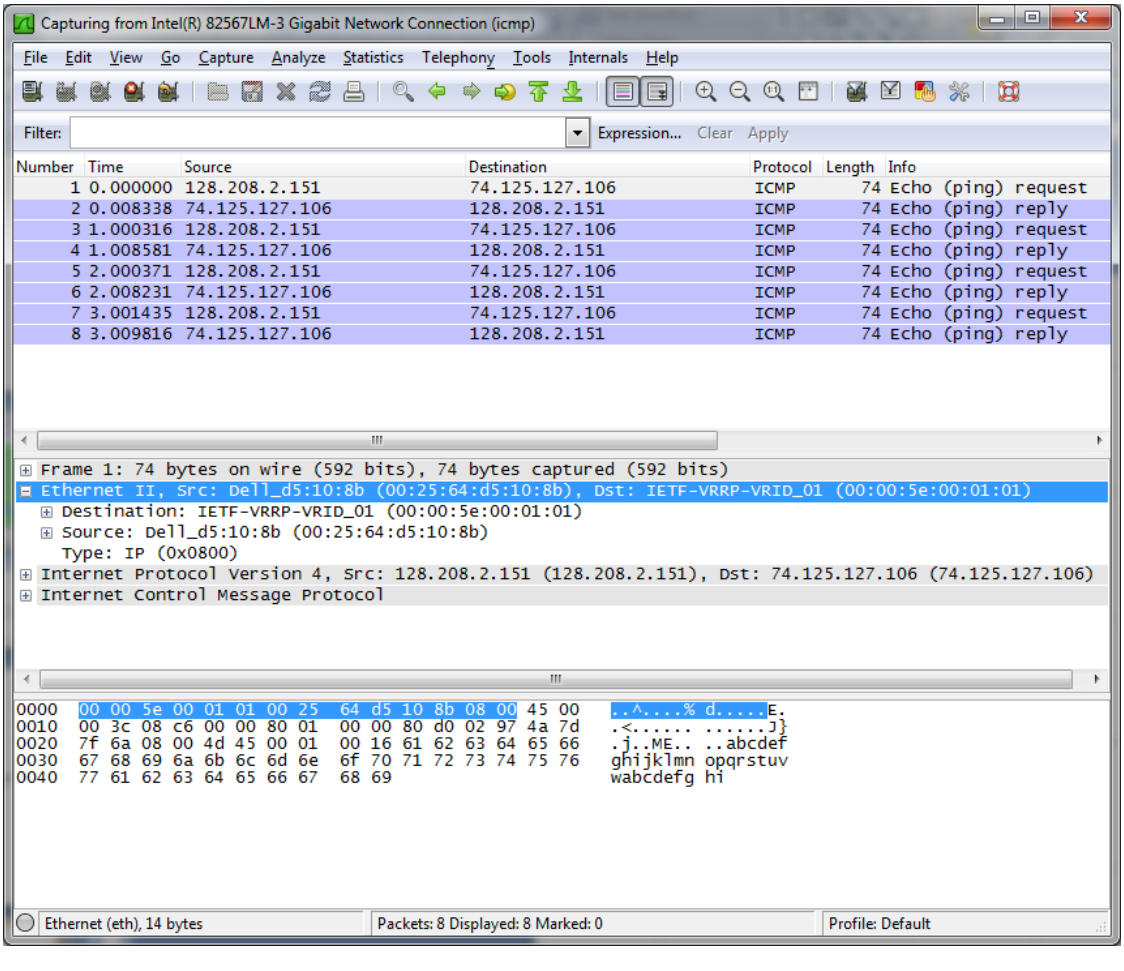


Figure 6: Trace of ping traffic, showing Ethernet details of the first packet

Step 2: Inspect the Trace

Select any packet in the trace (in the top panel) to see details of its structure (in the middle panel) and the bytes that make up the packet (in the bottom panel). Now we can inspect the details of the packets. Note that we are using the term "packet" in a loose way. Each record captured by Wireshark more correctly corresponds to a single frame in Ethernet format that carries a packet as its payload; Wireshark interprets as much structure as it can.

In the middle panel, expand the Ethernet header fields (using the "+" expander or ^ icon) to see their details. Our interest is the Ethernet header, and you may ignore the higher layer protocols (which are IP and ICMP in this case). You can click on the Ethernet header to see the bytes that correspond to it in the packet highlighted in the bottom panel.

Compare the fields you see with the picture of an Ethernet frame in Fig. 4-14 of your text.

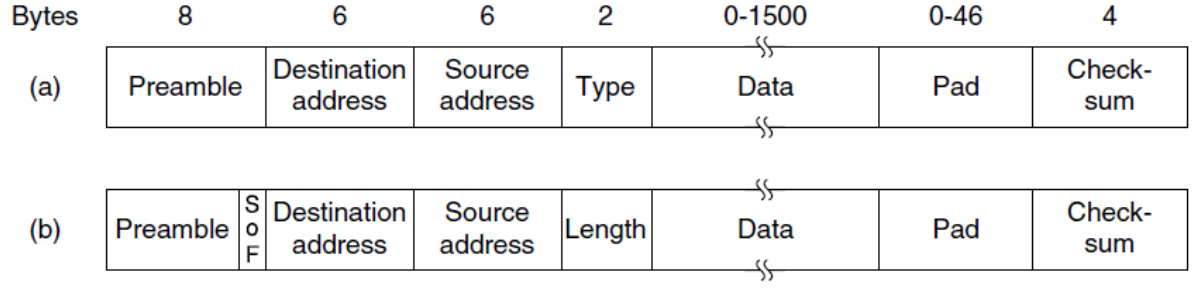


Figure 4-14. Frame formats. (a) Ethernet (DIX). (b) IEEE 802.3.

You will see both similarities and differences:

- There are two kinds of Ethernet shown in the book, IEEE 802.3 and DIX Ethernet. IEEE 802.3 is rare and you are not likely to see it. The frames in the figure and likely your capture are DIX (Digital, Intel, Xerox) Ethernet, called "Ethernet II" in Wireshark.
- There is no preamble in the fields shown in Wireshark. The preamble is a physical layer mechanism to help the NIC identify the start of a frame. It carries no useful data and is not received like other fields.
- There is a destination address and a source address. Wireshark is decoding some of these bits in the OUI (Organizationally Unique Identifier) portion of the address to tell us the vendor of the NIC, e.g., Dell for the source address.
- There is a Type field. For the ping messages, the Ethernet type is IP, meaning the Ethernet payload carries an IP packet. (There is no Length field as in the IEEE 802.3 format. Instead, the **length of a DIX Ethernet frame is determined by the hardware of a receiving computer**, which looks for valid frames that start with a preamble and end with a correct checksum, and are passed up to higher layers along with the packet.)
- There is no Data field – the data starts with the IP header right after the Ethernet header.
- There is no pad. A pad will be present at the end, if the frame would otherwise be less than 64 bytes, the minimum Ethernet frame size.
- There is no checksum in most traces, even though it really does exist. Typically, Ethernet hardware that is sending or receiving frames computes or checks this field and adds or strips it. Thus, it is simply not visible to the OS or Wireshark in most capture setups.
- There are also no VLAN fields such as the Tag shown in Fig. 4-49. If VLANs are in use, the VLAN tags are normally added and removed by switch ports so they will not be visible at host computers using the network.

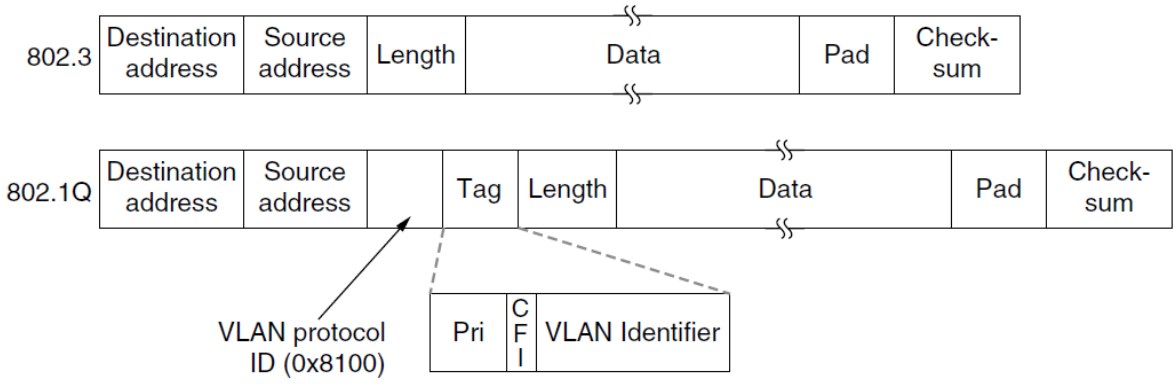


Figure 4-49. The 802.3 (legacy) and 802.1Q Ethernet frame formats.

Step 3: Ethernet Frame Structure

Question 3 (5 marks): To show your understanding of the Ethernet frame format by drawing a figure of the ping message that shows the position and "total size" in bytes of the Ethernet header fields. Your figure can simply show the frame as a long, thin rectangle. The leftmost fields come first in the packet and are sent on the wire first. On this drawing, show the range of the Ethernet header and the Ethernet payload. Add a field at the end to represent the 4-byte checksum; we know it is there even if Wireshark does not show us this field.
To work out sizes, observe that when you click on a protocol block in the middle panel (the block itself, not the "+" expander) then Wireshark will highlight the bytes it corresponds to in the packet in the lower panel and display the length at the bottom of the window. You may also use the overall packet size shown in the Length column or Frame detail block.

Also, print the related details referred to in question 3 above. To do so, select Print from the Wireshark File command menu, and select the "Selected Packet Only" and "Print as displayed" radial buttons, and then click OK.

Step 4: Broadcast Frames

The trace that you gathered above captured unicast Ethernet traffic sent between a specific source and destination, e.g., your computer to the router. It is also possible to send multicast or broadcast Ethernet traffic, destined for a group of computers or all computers on the Ethernet, respectively. We can tell from the address whether it is unicast, multicast, or broadcast. Broadcast traffic is sent to a reserved Ethernet address that has all bits set to "1". Multicast traffic is sent to addresses that have a "1" in the first bit sent on the wire; broadcast is a special case of multicast. Broadcast and multicast traffic is widely used for discovery protocols, e.g., a packet sent to everyone in an effort to find the local printer. *Start a capture for broadcast and multicast Ethernet frames with a filter of* "ether multicast", *wait up to 30 seconds to record background traffic, and then stop the capture. If you do not capture any packets with this filter then use the trace that we supplied.* On most Ethernets, there is a steady chatter of background traffic as computers exchange messages to maintain network state, which is why we try to **capture traffic without running any other programs**. The capture filter of "ether multicast" will capture both multicast and broadcast Ethernet frames, but not regular unicast frames. You may have to wait a little while for these packets to be captured, but on most LANs with multiple computers, you will see at least a packet every few seconds.

Examine the multicast and broadcast packets that you captured, looking at the details of the source and destination addresses. Most likely one has the broadcast Ethernet address, as broadcast frames tend to be more common than multicast frames. Look at a broadcast frame to see what address is used

for broadcast by Ethernet. Expand the Ethernet address fields of either broadcast or multicast frames to see which bit is set to distinguish broadcast/multicast or group traffic from unicast traffic.

Now, answer the following questions:

Question 4 (1 mark): What is the broadcast Ethernet address, written in standard form as Wireshark displays it?

Question 5 (3 marks): Which bit of the Ethernet address is used to determine whether it is unicast or multicast/broadcast? Also, what values of this bit correspond to unicast or multicast/broadcast?

Congratulations! You now understand the basics of Wireshark!

The goal of this first section was primarily to introduce you to Wireshark. The five questions above will demonstrate that you've been able to get Wireshark up and running and have explored some of its capabilities. Answer the questions, based on your Wireshark experimentation.